

Coding & Solution Implementation Report

Date:	08 October 2026	Team ID:	UNM1611JASON
Project Name:	AI BlogNest API	Maximum Marks:	5 Marks

Solution Implementation Overview:

This section documents the completeness of the implemented AI BlogNest API codebase, demonstrating clean code practices, working features, and adherence to requirements.

Solution Summary:

Repository Link / URL:	https://github.com/sharvesh007ram/ai-blognest-api (Local deliverables included)
Programming Language(s):	JavaScript (Node.js ES6+ Async/Await)
Frameworks & Libraries:	Express.js (v4.18.2), Mongoose (v7.5.0), @google/generative-ai (v0.24.1), jsonwebtoken, bcryptjs
Key Features Implemented:	• User registration with bcrypt hashing & JWT token issuance • Full Blog CRUD API with author ownership enforcement • Gemini 1.5 Flash Blog Generation with regex markdown cleanup • Gemini text summarizer producing single executive paragraphs • Dynamic topic category inferencing across 9 domains • Centralized error handling and Thunder Client verification suite
Pending Features:	None. 100% of planned functional and AI capabilities are implemented and tested.
Setup / Run Instructions:	1. Clone repository & run npm install 2. Create .env with PORT=5000, MONGO_URI, JWT_SECRET, GEMINI_API_KEY 3. Execute npm run dev or npm start 4. Import thunder-client-ai-blognest-api.postman_collection.json to test

Coding & Solution — Quality Checklist & Code Verification

Code Quality Checklist:

S.No	Criteria	Status	Verification Evidence in AI BlogNest API
1	Code is modular and organized into functions/classes	Yes	Clean separation: controllers/aiController.js, services/geminiService.js, models/Blog.js
2	Meaningful variable and function names are used	Yes	Variables: existingUser, resolvedCategory, prompt, summaryText; Functions: createBlog, callGemini
3	Code includes comments/documentation where necessary	Yes	Comments explain fallback to gemini-1.5-flash if env model is missing, regex text cleaning rules
4	Error handling is implemented for critical operations	Yes	Validation checks with express-validator, try/catch blocks routing to errorMiddleware
5	The application runs without critical errors	Yes	Verified live: connected to MongoDB Atlas, listening on port 5000, 100% tests passing
6	Code is committed to a version control repository	Yes	Structured with git, .gitignore excluding node_modules and .env secrets

Technical Architecture Highlights:

- Prompt Engineering Pipeline:** "Write a blog post about \${topic}. Include a strong introduction, key points of engaging content, and a conclusion. Do not include markdown headings, asterisks, or extra formatting. Give final output in only 100-150 words."
- Author Guard Middleware:** Verifies that blog.author.toString() === req.user._id.toString() before allowing PUT or DELETE mutations, returning a 403 Forbidden on violation.
- Text Sanitizer Regex:** `text.replace(/^[s*#{1,6}]s*/gm, "").replace(/^[s*[\*-+]]s*/gm, "").replace(/^{1,3}(.+?)^{1,3}/g, '$1')` ensures pristine article display.